



Politechnika Wrocławska

Sprawozdanie: Mikroprojekt 2

Grafy

Grupa 4 | Prowadzący: dr inż. Krzysztof Halawa

Jan Nowak

284138

1 Krótkie wprowadzenie

Celem projektu była implementacja gry logicznej wykorzystującej techniki sztucznej inteligencji. Wybrano grę w „Kółko i krzyżyk”, rozbudowaną o w pełni funkcjonalny interfejs graficzny. Gra pozwala na rozgrywkę w dwóch trybach: Gracz vs Komputer (AI) oraz Gracz vs Gracz.

2 Opis tworzonej gry

Aplikacja to klasyczna implementacja gry „Kółko i krzyżyk” zrealizowana w języku C++. Interfejs graficzny został napisany z użyciem biblioteki **SDL2** oraz rozszerzenia **SDL_ttf** do renderowania tekstu. Plansza gry ma stały rozmiar 3×3 , a celem gracza jest ułożenie trzech swoich znaków w rzędzie, kolumnie lub po przekątnej.

Aplikacja oferuje cztery poziomy trudności w trybie dla pojedynczego gracza: „Łatwy”, „Średni”, „Trudny” oraz „Niemożliwy”. W prawym panelu wyświetlane są bieżące statystyki rozgrywki (wygrane poszczególnych stron oraz remisy). Dodatkowo zaimplementowano wizualne podświetlanie wygrywającego segmentu, aby ułatwić odczytywanie wyniku zakończenia rundy.

3 Wytlumaczenie stosowanych technik SI

Głównym mechanizmem decyzyjnym sztucznej inteligencji w aplikacji jest algorytm **MinMax** rozszerzony o optymalizację w postaci cięcia alfa-beta. Algorytm ten służy do wyznaczania optymalnego ruchu w grach dwuosobowych o sumie zerowej z pełną informacją.

3.1 Algorytm MinMax i cięcie alfa-beta

W zaimplementowanej funkcji `minmax()`, algorytm rekurencyjnie analizuje drzewo możliwych stanów gry. Każdemu stanowi planszy przypisywana jest ocena rozgrywki:

- **Wygrana AI:** Algorytm zwraca wynik "10 – głębokość". Dzięki odjęciu głębokości (liczby ruchów potrzebnych do osiągnięcia celu), komputer preferuje szybsze wygrane.
- **Wygrana Gracza:** Algorytm zwraca "głębokość – 10". Gwarantuje to, że w przypadku nieuniknionej przegranej, sztuczna inteligencja wybierze ścieżkę jak najpóźniejszej porażki.
- **Remis (lub plansza pełna):** Zwracana jest wartość 0.

```
//Algorytm MinMax
static int minmax(Plansza& plansza, int glebokosc, bool maksymalizuje, int alfa, int beta) {
    char wynik = sprawdzWygrana(plansza);
    if (wynik == AI){
        return 10 - glebokosc;
    }
    if (wynik == GRACZ){
        return glebokosc - 10;
    }
    if (czyPelna(plansza)){
        return 0;
    }
    int najlepszy = maksymalizuje ? INT_MIN : INT_MAX;
    for (int r = 0; r < ROZM_PLAN; r++) {
        for (int c = 0; c < ROZM_PLAN; c++) {
            if (plansza[r][c] != PUSTE){
                continue;
            }
            plansza[r][c] = maksymalizuje ? AI : GRACZ;
            int ocena = minmax(plansza, glebokosc + 1, !maksymalizuje, alfa, beta);
            plansza[r][c] = PUSTE;
            if (maksymalizuje) {
                najlepszy = max(najlepszy, ocena);
                alfa = max(alfa, najlepszy);
            } else {
                najlepszy = min(najlepszy, ocena);
                beta = min(beta, najlepszy);
            }
            if (beta <= alfa){
                goto przytnij;
            }
        }
    }
    przytnij:
    return najlepszy;
}
```

Rysunek 1: Implementacja algorytmu MinMax

Gracz w symulacji (Max) stara się maksymalizować wynik gry, natomiast przeciwnik (Min) stara się go zminimalizować. Aby ograniczyć liczbę odwiedzanych węzłów i zoptymalizować czas działania, zastosowano odcięcie alfa-beta, gdzie:

- α – najlepszy wynik, jaki maksymalizator może zagwarantować.
- β – najlepszy wynik, jaki minimalizator może zagwarantować.

Gdy podczas przeszukiwania gałęzi drzewa spełniony zostanie warunek $\beta \leq \alpha$, pętla iterująca po wolnych polach jest przerywana (`goto przytnij`), omijając stany, które nie zostałyby wybrane przez optymalnie grających przeciwników.

3.2 Skalowanie poziomu trudności

W funkcji `wybierzRuchAI` poziom trudności jest reprezentowany przez wartość zmiennej (od 0.20 dla poziomu Łatwego do 1.00 dla Niemożliwego). Na początku tury AI losowana jest wartość z przedziału (0, 1). Jeśli przewyższa ona przypisany próg trudności, komputer wykonuje losowy ruch. W przeciwnym razie wywołany jest algorytm MinMax wyliczający ruch idealny. Przykładowo jeśli poziom trudności jest ustawiony na **Trudny** to wartość trudności jest równa 0.85 i istnieje 15% szans na postawienie kółka na losowej pozycji.

```
static pair<int, int> wybierzRuchAI(Plansza plansza, double trudnosc){
    vector<pair<int, int>> wolnePola;
    for (int r = 0; r < ROZM_PLAN; r++){
        for (int c = 0; c < ROZM_PLAN; c++){
            if (plansza[r][c] == PUSTE){
                wolnePola.push_back({r, c});
            }
        }
    }

    double losowa = (double)rand() / RAND_MAX;
    if (losowa > trudnosc)
        return wolnePola[rand() % wolnePola.size()];
    int najlepszyWynik = INT_MIN;
    pair<int, int> najlepszyRuch = wolnePola[0];
    for (auto [r, c] : wolnePola) {
        plansza[r][c] = AI;
        int ocena = minmax(plansza, 0, false, INT_MIN, INT_MAX);
        plansza[r][c] = PUSTE;
        if (ocena > najlepszyWynik) {
            najlepszyWynik = ocena;
            najlepszyRuch = {r, c};
        }
    }
    return najlepszyRuch;
}
```

Rysunek 2: Funkcja `wybierzRuchAI`

4 Podsumowanie i wnioski

- Zastosowanie algorytmu MinMax z cięciem alfa-beta pozwoliło na bezbłędne wyznaczanie ruchów przez komputer przy jednoczesnym ograniczeniu liczby przeszukiwanych stanów gry i skróceniu czasu działania aplikacji.
- Mechanizm oparty na prawdopodobieństwie wykonania ruchu losowego zamiast ruchu idealnego pozwolił na płynne i efektywne dostosowanie poziomu trudności do gracza.

5 Bibliografia

- <https://github.com/Cledersonbc/tic-tac-toe-minimax>
- <https://www.geeksforgeeks.org/dsa/finding-optimal-move-in-tic-tac-toe-using-minimax-algorithm-in-game-theory/>.
- <https://www.youtube.com/watch?v=2Tr8LkyU78c>